Week 3

I. Web & Mobile Application Architecture & System Design

(To be completed during system design and planning)

- **System Overview & Workflow (Client \rightarrow Server \rightarrow Model Inference):**
- **Frontend Architecture (Web UI via React.js/Vite & Mobile UI via Responsive Web, PWA, or React Native/Flutter):**
  - *Cross-Platform Layout & Responsive UI Strategy:*
- **Backend Architecture (Node.js, Express.js RESTful API & JSON/Multipart Payload Handling):**
  - *API Endpoints Designed for Web & Mobile Clients (e.g., /api/predict, /api/health, /api/upload):*
- **Mobile-Specific Considerations (Touch controls, microphone/file access permissions, offline caching, and bandwidth optimization):**

II. Model Hosting & Storage Strategy

(To be completed before deploying model weights)

- **Model Weight Storage Strategy (Hugging Face Hub / Cloud Bucket integration for large files):**
- **Model Inference Setup (Local Python worker, TorchScript, or ONNX runtime integration with Node.js):**
- **Security & Environment Configuration (Handling API keys, .env files, CORS configurations for mobile/web clients):**

III. Practical Implementation & Verification Checklist

(Hands-on execution log for Week 3: Full-Stack & Mobile-Ready Architecture)

- [ ] **Frontend & Mobile UI Component Wireframe:** Designed and implemented responsive layouts supporting both desktop web browsers and mobile device viewports (touch-friendly controls, camera/audio upload wrappers).
- [ ] **Backend API Development:** Configured Express.js server in Node.js with CORS and payload limits to accept requests from both web and mobile clients.
- [ ] **Model Integration & Inference Pipeline:** Connected backend server to the trained model script/runtime; verified that mobile/web requests successfully trigger inference and return structured JSON payloads.
- [ ] **Model Weight Hosting & Cloud Retrieval:** Uploaded trained model checkpoints to Hugging Face Hub / remote storage; implemented automated download/caching logic on server startup.
- [ ] **Cross-Platform End-to-End Testing:** Validated full communication flow (Mobile/Web Client UI \rightarrow Express Backend \rightarrow PyTorch/ONNX Model Inference

\rightarrow Client Display) across simulated mobile devices or responsive viewports.

IV. Implementation Notes & Code Snippets

- **Mobile UI / Cross-Platform Integration Challenges Faced:**
- **Core API Endpoint or Mobile-Compatible Request Handler Snippet:**

```
// Paste Express route or mobile client fetch handler here
```